

DSAA2043: Exercise 3

Notes

- This exercise is for practice only and does not count toward the final score.
- Both 0-indexed and 1-indexed arrays are accepted. However, when using Python, array indices must start from 0.

Problem 1. Stacks are dynamic sets that follow the Last In, First Out (LIFO) principle, meaning the last element added to the stack is the first one to be removed. They are analogous to a stack of plates, where you can only add or remove plates from the top. Common operations on a stack include PUSH (to add an element), POP (to remove the top element), and PEEK (to view the top element without removing it).

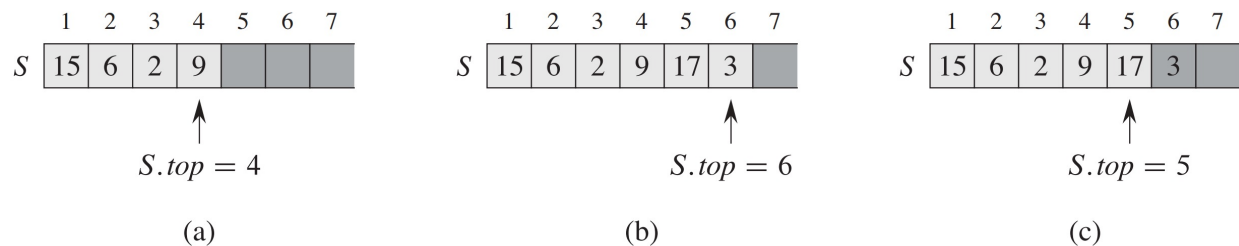


Figure 1: An array implementation of a Stack S . Stack elements appear only in the lightly shaded positions. (a) Stack S has 4 elements. The top element is 9. (b) Stack S after the calls $PUSH(S, 17)$ and $PUSH(S, 3)$. (c) Stack S after the call $POP(S)$ has returned the element 3, which is the one most recently pushed. Although element 3 still appears in the array, it is no longer in the stack; the top is element 17.

Answer the following questions:

- Q1.1.** Using Figure 1 as a reference, illustrate the result of each operation in the sequence $PUSH(S, 4)$, $PUSH(S, 1)$, $PUSH(S, 3)$, $POP(S)$, $PUSH(S, 8)$, and $POP(S)$ on an initially empty stack S stored in array $S[1..6]$.
- Q1.2.** Provide a pseudocode or Python implementation of the Stack operations including $IF_EMPTY()$, $PUSH()$ and $POP()$. What is the time complexity of the $PUSH$ and POP operations?
- Q1.3.** Explain how to implement two stacks in one array $A[1..n]$ in such a way that neither stack overflows unless the total number of elements in both stacks together is n . The $PUSH$ and POP operations should run in $O(1)$ time.
- Q1.4.** Show how to implement a queue using two Stacks (Provide a pseudocode or Python implementation). Analyze the running time of the queue operations.

Answer 1.1: Below is the step-by-step result of each operation on the given Stack S :

Index	0	1	2	3	4	5
Initial						
Push(S,4)	4					
Push(S,1)	4	1				
Push(S,3)	4	1	3			
Pop(S)	4	1				
Push(S,8)	4	1	8			
Pop(S)	4	1				

Answer 1.2:

Pseudocode:

Algorithm 1 Stack Operations

```

1: function IF-EMPTY( $S$ )
2:   if  $S.top = 0$  then
3:     return TRUE
4:   else
5:     return FALSE
6:   end if
7: end function
8:
9: function PUSH( $S, x$ )
10:  if  $S.top = S.size$  then
11:    error "Stack overflow"
12:  end if
13:   $S.top \leftarrow S.top + 1$ 
14:   $S.array[S.top] \leftarrow x$ 
15: end function
16:
17: function POP( $S$ )
18:  if IF-EMPTY( $S$ ) then
19:    error "Stack underflow"
20:  end if
21:   $S.top \leftarrow S.top - 1$ 
22:  return  $S.array[S.top + 1]$ 
23: end function

```

Python Implementation (Array-like):

```

class Stack:
    def __init__(self, size):
        self.array = [None] * size
        self.top = 0
        self.size = size

    def if_empty(self):
        return self.top == 0

```

```

def push(self, item):
    if self.top == self.size:
        print("Stack overflow")
        return None
    self.top += 1
    self.array[self.top - 1] = item

def pop(self):
    if self.is_empty():
        print("Stack underflow")
        return None
    self.top -= 1
    return self.array[self.top]

```

Python Implementation (List-like):

```

class Stack:
    def __init__(self):
        self.stack = []

    def is_empty(self):
        return len(self.stack) == 0

    def push(self, item):
        self.stack.append(item)

    def pop(self):
        # Remove and return the top item of the stack
        if self.is_empty():
            print("Stack Underflow: Cannot pop from an empty stack.")
            return None
        return self.stack.pop()

```

Time Complexity of the PUSH and POP operations:

- PUSH(): $O(1)$ - constant time, just increments the top pointer and assigns a value.
- POP(): $O(1)$ - constant time, just decrements the top pointer and returns a value

Answer 1.3: We use a single array $A[1..n]$ to store elements for two stacks. The implementation is described below:

1. Division of the Array:

- The array is logically divided into two parts:
 - **Stack 1:** Grows from the *left* (starting at index 1) towards the *right*.
 - **Stack 2:** Grows from the *right* (starting at index n) towards the *left*.

2. Pointers:

- Two pointers are used to track the topmost elements of the two stacks:
 - **top1**: Points to the topmost element of **Stack 1**. Initially set to 0 (indicating Stack 1 is empty).
 - **top2**: Points to the topmost element of **Stack 2**. Initially set to $n + 1$ (indicating Stack 2 is empty).

3. Key Condition:

- The two stacks can grow until the condition:

$$\text{top1} + 1 = \text{top2}$$

At this point, the array is fully occupied, and no more elements can be pushed into either stack without causing overflow.

Python Implementation:

```
class TwoStacks:
    def __init__(self, n):
        """
        Initialize an array of size n to hold two stacks.
        """
        self.size = n
        self.array = [None] * n # Shared array for both stacks
        self.top1 = -1          # Top of Stack 1 starts at the beginning
        self.top2 = n           # Top of Stack 2 starts at the end

    def push1(self, item):
        """
        Push an item onto Stack 1.
        """
        if self.top1 + 1 < self.top2: # Check if there's space
            self.top1 += 1
            self.array[self.top1] = item
        else:
            print("Stack Overflow for Stack 1")

    def push2(self, item):
        """
        Push an item onto Stack 2.
        """
        if self.top1 + 1 < self.top2: # Check if there's space
            self.top2 -= 1
            self.array[self.top2] = item
        else:
            print("Stack Overflow for Stack 2")

    def pop1(self):
        """
```

```

    Pop an item from Stack 1.
    """
    if self.top1 >= 0: # Check if Stack 1 is not empty
        item = self.array[self.top1]
        self.top1 -= 1
        return item
    else:
        print("Stack Underflow for Stack 1")
        return None

def pop2(self):
    """
    Pop an item from Stack 2.
    """
    if self.top2 < self.size: # Check if Stack 2 is not empty
        item = self.array[self.top2]
        self.top2 += 1
        return item
    else:
        print("Stack Underflow for Stack 2")
        return None

```

Answer 1.4: A queue operates on the FIFO (First In First Out) principle, while a stack operates on the LIFO (Last In First Out) principle. To implement a queue using two stacks, we can simulate the FIFO behavior by carefully moving elements between the stacks.

The main idea is as following:

1. Two Stacks Used:

- **stack1:** Used for enqueue operations (adding elements).
- **stack2:** Used for dequeue operations (removing elements).

2. Enqueue Operation:

- Push the element into **stack1**.

3. Dequeue Operation:

- If **stack2** is empty:
 - Pop all elements from **stack1** and push them into **stack2** (this reverses their order).
- Pop the top element from **stack2** (this represents the front of the queue).
- If both **stack1** and **stack2** are empty, the queue is empty.

Python Implementation:

```

class QueueUsingTwoStacks:
    def __init__(self):
        self.stack1 = [] # Stack for enqueue
        self.stack2 = [] # Stack for dequeue

```

```

def enqueue(self, item):
    self.stack1.append(item)

def dequeue(self):
    """
    Remove and return the front item from the queue.
    If the queue is empty, return None.
    """
    if not self.stack2:
        while self.stack1: # Move all elements from stack1 to stack2
            self.stack2.append(self.stack1.pop())

    if self.stack2: # If stack2 is not empty, pop the top element
        return self.stack2.pop()
    else:
        print("Queue is empty!")
        return None

def if_empty(self):
    return not self.stack1 and not self.stack2

def size(self):
    return len(self.stack1) + len(self.stack2)

```

Pseudocode:

Algorithm 2 Queue Implementation Using Two Stacks

```

1: Initialize: Two empty stacks: stack1 and stack2
2: function ENQUEUE(element)
3:   Push element onto stack1
4: end function
5:
6: function DEQUEUE
7:   if stack2 is empty then
8:     while stack1 is not empty do
9:       Pop the top element from stack1
10:      Push it onto stack2
11:     end while
12:   end if
13:   if stack2 is empty then
14:     return Error: Queue is empty
15:   else
16:     Pop the top element from stack2 and return it
17:   end if
18: end function

```

Time Complexity of the ENQUEUE and DEQUEUE operations:

- ENQUEUE(): $O(1)$ - Pushing an element into stack1 takes $O(1)$.
- DEQUEUE(): $O(1)$ - If stack2 is not empty, popping from stack2 takes $O(1)$. If stack2 is empty, we need to move all elements from stack1 to stack2. This takes $O(n)$, where n is the number of elements in stack1. However, once the elements are moved, subsequent dequeue operations are $O(1)$ until stack2 is empty again. Amortized Time Complexity = $O(1)$.

Problem 2. Suppose that an intermixed sequence of push and pop operations are performed. The pushes push the integers 0 through 9 in order; the pops print out the return value. Which of the following sequences could not occur?

- (a) 4 3 2 1 0 9 8 7 6 5
- (b) 4 6 8 7 5 3 2 9 0 1
- (c) 2 5 6 7 4 8 9 3 1 0
- (d) 4 3 2 1 0 5 6 7 8 9

Answer 2: To determine which of the sequences could not occur, it is necessary to analyze whether the sequence adheres to the constraints of a Stack data structure. A Stack operates in a LIFO (last-in, first-out) manner, meaning the most recently pushed element must be the first to be popped. Steps to analyze each sequence are as follows:

1. Push the integers 0 through 9 **in order**.
2. Track the sequence of pops and verify if it respects the LIFO rule.
3. If the sequence violates the stack's behavior, it is an invalid sequence.

Thus we analyze the validity of each sequence one by one:

(a) 4 3 2 1 0 9 8 7 6 5

1. Push 0 to 4 onto the stack, then pop them in reverse order: 4 3 2 1 0. This is valid.
2. Push 5 to 9 onto the stack, then pop them in reverse order: 9 8 7 6 5. This is valid.
3. This sequence respects the LIFO rule. **Valid**.

(b) 4 6 8 7 5 3 2 9 0 1

1. Push 0 to 4 onto the stack, then pop 4. This is valid.
2. Push 5 and 6, then pop 6. This is valid.
3. Push 7 and 8, then pop 8 and 7. This is valid.
4. Pop 5, 3 and 2. This is valid.
5. Push 9 and Pop 9. This is valid.
6. Pop 0 and 1. This violates the LIFO rule because 0 was pushed before 1, but 1 was popped after 0. This sequence cannot occur. **Invalid**.

(c) 2 5 6 7 4 8 9 3 1 0

1. Push 0 to 2, then pop 2. This is valid.
2. Push 3 to 5, then pop 5. This is valid.
3. Push 6 and Pop 6. This is valid.

4. Push 7 and Pop 7. This is valid.
5. Pop 4. This is valid.
6. Push 8 and Pop 8. This is valid.
7. Push 9 and Pop 9. This is valid.
8. Pop remaining 3, 1 and 0. This sequence respects the LIFO rule. **Valid.**

(d) 4 3 2 1 0 5 6 7 8 9

1. Push 0 to 4, then Pop all in reverse (4,3,2,1,0). This is valid.
2. For 5 to 9, Push and immediately Pop. This is valid.
3. This sequence respects the LIFO rule. **Valid.**

Problem 3. Queues are dynamic sets that follow the First In, First Out (FIFO) principle, meaning the first element added to the queue is the first one to be removed. They are analogous to a line of people waiting, where new arrivals join at the end of the line, and service is provided to those at the front. Common operations on a queue include Enqueue (to add an element to the rear), Dequeue (to remove the front element), and Peek (to view the front element without removing it). Queues are widely used in programming for tasks such as scheduling, buffering, and managing resources in a sequential manner.

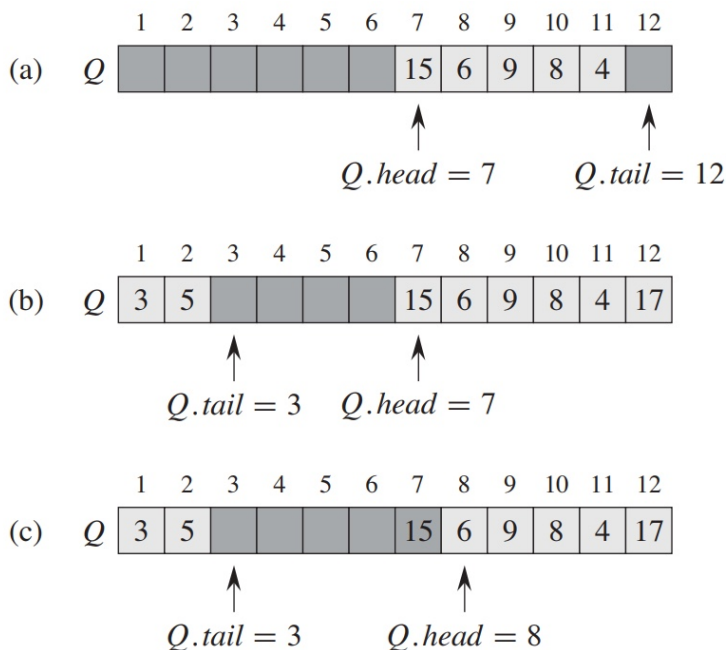


Figure 2: A Queue implemented using an array $Q[1..12]$. Queue elements appear only in the lightly shaded positions. (a) The queue has 5 elements, in locations $Q[7..11]$. (b) The configuration of the queue after the calls $ENQUEUE(Q, 17)$, $ENQUEUE(Q, 3)$, and $ENQUEUE(Q, 5)$. (c) The configuration of the queue after the call $DEQUEUE(Q)$ returns the key value 15 formerly at the head of the queue. The new head has key 6.

Answer the following questions:

- Q3.1.** Using Figure 2 as a reference, illustrate the result of each operation in the sequence $ENQUEUE(Q, 4)$, $ENQUEUE(Q, 1)$, $ENQUEUE(Q, 3)$, $DEQUEUE(Q)$, $ENQUEUE(Q, 8)$, and $DEQUEUE(Q)$, on an initially empty queue Q stored in array $Q[1..6]$.
- Q3.2.** Provide a pseudocode or Python implementation of the Queue operations including $ENQUEUE()$ and $DEQUEUE()$. What is the time complexity of the $ENQUEUE()$ and $DEQUEUE()$ operations?
- Q3.3.** Show how to implement a stack using two Queues (Provide a pseudocode or Python implementation). Analyze the running time of the stack operations.

Answer 3.1: Below is the step-by-step result of each operation on the given Queue S :

Index	0	1	2	3	4	5
Initial						
Enqueue(Q,4)	4					
Enqueue(Q,1)	4	1				
Enqueue(Q,3)	4	1	3			
Dequeue(Q)	1	3				
Enqueue(Q,8)	1	3	8			
Dequeue(Q)	3	8				

Answer 3.2:

Pseudocode:

Algorithm 3 Queue Operations

```

1: function IS-EMPTY( $Q$ )
2:   if  $Q.front = Q.rear$  then
3:     return TRUE
4:   else
5:     return FALSE
6:   end if
7: end function
8:
9: function ENQUEUE( $Q, x$ )
10:  if  $Q.rear = Q.size$  then
11:    error "Queue overflow"
12:  end if
13:   $Q.array[Q.rear] \leftarrow x$ 
14:   $Q.rear \leftarrow Q.rear + 1$ 
15: end function
16:
17: function DEQUEUE( $Q$ )
18:  if IS-EMPTY( $Q$ ) then
19:    error "Queue underflow"
20:  end if
21:   $x \leftarrow Q.array[Q.front]$ 
22:   $Q.front \leftarrow Q.front + 1$ 
23:  return  $x$ 
24: end function

```

Python Implementation:

```

class Queue:
    def __init__(self):
        # Initialize an empty list to represent the queue
        self.queue = []

    def Enqueue(self, x):
        """Add an element to the end of the queue."""

```

```

        self.queue.append(x)

def Dequeue(self):
    """Remove and return the front element of the queue."""
    if self.IsEmpty():
        return "Queue is empty"
    return self.queue.pop(0) # Remove the first element

def IsEmpty(self):
    """Check if the queue is empty."""
    return len(self.queue) == 0

```

Time Complexity of the ENQUEUE and DEQUEUE operations:

- ENQUEUE(): $O(1)$ - constant time, appending an element to the end of a list takes $O(1)$ time.
- DEQUEUE(): $O(n)$ - Removing the first element in a list (`pop(0)`) requires shifting all subsequent elements one step forward, which takes $O(n)$ time, where n is the number of elements in the queue.

Note: If we want to optimize the Dequeue() operation to achieve $O(1)$ time complexity, we can implement the queue using a deque (double-ended queue) from Python's collections module:

```

from collections import deque

class QueueOptimized:
    def __init__(self):
        # Initialize an empty deque
        self.queue = deque()

    def Enqueue(self, x):
        """Add an element to the end of the queue."""
        self.queue.append(x) #  $O(1)$ 

    def Dequeue(self):
        """Remove and return the front element of the queue."""
        if self.IsEmpty():
            return "Queue is empty"
        return self.queue.popleft() #  $O(1)$ 

    def IsEmpty(self):
        """Check if the queue is empty."""
        return len(self.queue) == 0

```

Answer 3.3: A stack operates on the LIFO (Last In First Out) principle, while a queue operates on the FIFO (First In First Out) principle. To implement a stack using two queues, we can simulate the LIFO behavior by carefully moving elements between the queues.

The main idea is as following:

1. Two Queues Used:

- **queue1**: Used as the main queue to store elements in stack order.
- **queue2**: Used as a temporary queue during Push operations.

2. Push Operation:

- Enqueue the new element into **queue2**.
- Dequeue all elements from **queue1** and enqueue them into **queue2**. This ensures the new element is at the front of **queue2**.
- Swap **queue1** and **queue2**, so **queue1** becomes the main queue with the correct stack order.

3. Pop Operation:

- Dequeue the front element from **queue1**. This represents the top of the stack.
- If **queue1** is empty, the stack is empty.

4. IsEmpty Operation:

- Check if **queue1** is empty. If it is, the stack is empty.

Pseudocode:

Algorithm 4 Stack Operations Using Two Queues

```

1: function PUSH( $S, x$ )
2:   Enqueue  $x$  into  $Q2$                                 ▷ Add the new element to Q2
3:   while  $Q1$  is not empty do                          ▷ Move all elements from Q1 to Q2
4:     Enqueue(Dequeue( $Q1$ )) into  $Q2$ 
5:   end while
6:   Swap  $Q1$  and  $Q2$                                     ▷ Q1 becomes the new main queue
7: end function
8:
9: function POP( $S$ )
10:  if  $Q1$  is empty then                                ▷ Check if the stack is empty
11:    error "Stack is empty"
12:  end if
13:  return Dequeue( $Q1$ )                                ▷ Remove and return the front of Q1
14: end function
15:
16: function ISEMPY( $S$ )
17:  return (Length( $Q1$ ) = 0)                            ▷ Return True if Q1 is empty, False otherwise
18: end function

```

Python Implementation:

```

class StackUsingLists:
    def __init__(self):
        # Initialize two lists to simulate queues
        self.Q1 = [] # Main queue
        self.Q2 = [] # Temporary queue

```

```

def Push(self, x):
    """Push an element onto the stack."""
    # Add the new element to Q2
    self.Q2.append(x)

    # Move all elements from Q1 to Q2
    while self.Q1:
        self.Q2.append(self.Q1.pop(0))

    # Swap Q1 and Q2 to maintain stack order
    self.Q1, self.Q2 = self.Q2, self.Q1

def Pop(self):
    """Remove and return the top element of the stack."""
    if not self.Q1:
        return "Stack is empty"
    return self.Q1.pop(0) # Remove and return the front element of Q1

def Top(self):
    """Get the top element of the stack without removing it."""
    if not self.Q1:
        return "Stack is empty"
    return self.Q1[0] # Return the front element of Q1

def IsEmpty(self):
    return len(self.Q1) == 0 # Return True if Q1 is empty, otherwise False

```

Time Complexity of the PUSH and POP operations:

- PUSH(): $O(n)$ - Moving all elements from queue1 to queue2 takes $O(n)$, where n is the current size of the stack.
- POP(): $O(1)$ - Dequeueing the front element of queue1 takes $O(1)$.

Problem 4. A palindrome is a word, phrase, or number that is spelled the same forward and backward. For example, “dad” is a palindrome; “A man, a plan, a canal: Panama” is a palindrome if you take out the spaces and ignore the punctuation; and 1,001 is a numeric palindrome. We can use a Stack to determine whether or not a given string is a palindrome.

Answer the following questions:

Q4.1. Provide pseudocode or Python code for an algorithm that takes a string of letters and returns True or False to determine whether it is palindromic.

Q4.2. Analyze the time and space complexity of your algorithm.

Answer 4.1: The key idea for implementing a solution to determine if a string is a palindrome using a stack is as follows:

1. **Preprocess the String:** Removing all non-alphanumeric characters from the string and converting it to lowercase.
2. **Store the First Half in a Stack:** Push the characters from the first half of the string onto a stack.
3. **Compare the Second Half with the Stack:** Starting from the middle of the string, compare each character from the second half with the characters popped from the stack.
4. **Determine the Result:** If all characters match, the string is a palindrome. Otherwise, it is not.

Pseudocode:

Algorithm 5 Check if a String is a Palindrome Using a Stack

```

1: function ISPALINDROME( $s$ )
2:   Remove all non-alphanumeric characters from  $s$  and convert it to lowercase.
3:   Initialize an empty stack  $stack$ .
4:   Let  $n \leftarrow$  length of  $s$ .
5:
6:   for  $i \leftarrow 0$  to  $\lfloor n/2 \rfloor - 1$  do                                 $\triangleright$  Push the first half of the string to the stack.
7:     Push  $s[i]$  onto  $stack$ .
8:   end for
9:
10:  for  $i \leftarrow \lceil n/2 \rceil$  to  $n - 1$  do                                 $\triangleright$  Compare the second half of the string with the stack.
11:    if  $stack$  is not empty and  $s[i] \neq \text{Pop}(stack)$  then
12:      return False
13:    end if
14:  end for
15:  return True
16: end function

```

Python Implementation:

```

def is_palindrome(s: str):
    # Step 1: Remove non-alphanumeric characters and convert to lowercase
    word = ''
    for char in s:
        if char.isalnum():
            word += char.lower()

    # Step 2: Initialize a stack
    stack = []
    n = len(word)

    # Step 3: Push the first half of the string onto the stack
    for i in range(n // 2):
        stack.append(word[i])

    # Step 4: Compare the second half of the string with the stack
    for i in range((n + 1) // 2, n): # Start from the middle or after the middle
        if stack and word[i] != stack.pop():
            return False

    # Step 5: If all characters match, return True
    return True

```

Answer 4.2: Time Complexity of the implemented algorithm:

- Preprocessing the string: Removing non-alphanumeric characters and converting to lowercase takes $O(n)$, where n is the length of the string.
- Pushing to the stack: For the first half of the string, we perform $n/2$ push operations, which take $O(n/2) = O(n)$.
- Comparing with the second half: For the second half of the string, we perform $n/2$ pop and comparison operations, which take $O(n/2) = O(n)$.
- **Total Time Complexity:** $O(n)$.

Space Complexity of the implemented algorithm:

- Stack: At most, we store half the characters of the string in the stack, which takes $O(n/2) = O(n)$.
- Other variables: The space required for preprocessing and other variables is negligible compared to the stack.
- **Total Space Complexity:** $O(n)$.

Problem 5. Given a string s representing an expression containing various types of brackets: $\{ \}$, $()$, and $[]$, provide pseudocode or Python code that reads in a string s , and determines whether the brackets in the expression are balanced or not. A balanced expression is one where every opening bracket has a corresponding closing bracket in the correct order.

Answer 5: To determine whether the brackets in a string are balanced, we can use a stack:

Python Implementation:

```
def check_brackets(string):
    left_brackets = 0
    right_brackets = 0
    stack = []

    bracket_pairs = {'(': ')', '[': ']', '{': '}'
    opening_brackets = '([{'
    closing_brackets = ')]}'

    for char in string:
        if char in opening_brackets:
            stack.append(char)
            left_brackets += 1
        elif char in closing_brackets:
            right_brackets += 1
            if len(stack) > 0 and stack[-1] == bracket_pairs[char]:
                stack.pop()

    print(left_brackets, right_brackets)
    print("YES" if not stack else "NO")
```